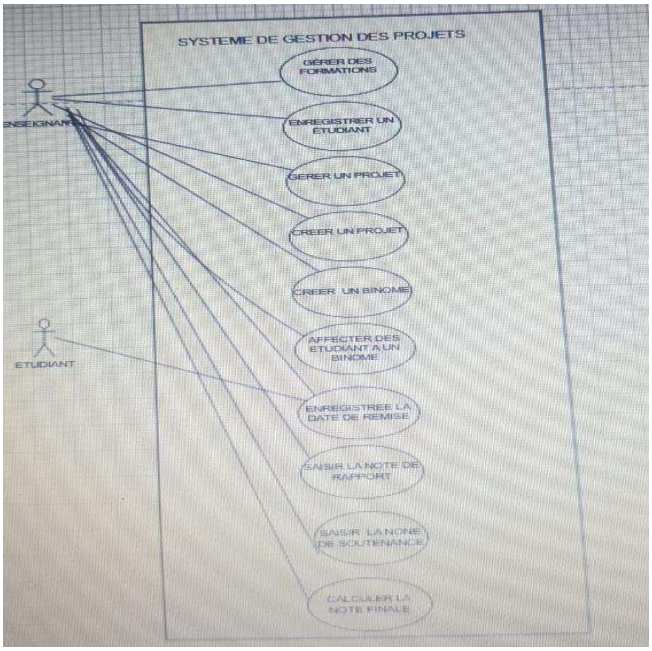
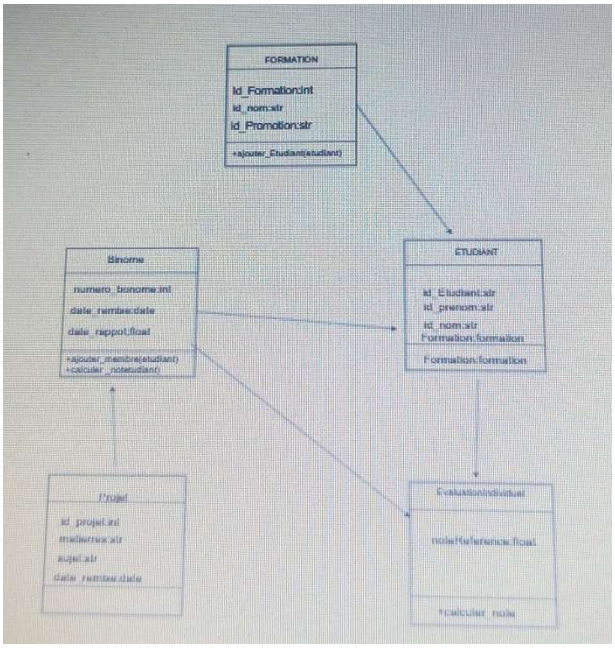
Projet UML et POO PYTHON

SUJET 1

1)**Diagramme de cas d'utilisation**



2) **Diagramme de classe**



3) **code python**

```
from __future__ import annotations

from datetime import date
```

```
class Formation:
    def __init__(self, id_formation: int, nom: str, promotion: str) -> None:
        self.__id_formation = id_formation
        self.__nom = nom
        self.__promotion = promotion
        self.__etudiants: list[Etudiant] = []

    def ajouter_etudiant(self, etudiant: "Etudiant") -> None:
        self.__etudiants.append(etudiant)

    def __str__(self) -> str:
        return f"Formation({self.__nom}, {self.__promotion})"

class Etudiant:
    def __init__(self, id_etudiant: int, nom: str, prenom: str, formation: Formation) -> None:
        self.__id_etudiant = id_etudiant
        self.__nom = nom
        self.__prenom = prenom
        self.__formation = formation
        formation.ajouter_etudiant(self)

    def __str__(self) -> str:
        return f"{self.__prenom} {self.__nom}"

class Projet:
    def __init__(self, id_projet: int, matiere: str, sujet: str, date_remise_prevue: date) -> None:
        self.__id_projet = id_projet
        self.__matiere = matiere
        self.__sujet = sujet
        self.__date_remise_prevue = date_remise_prevue
```

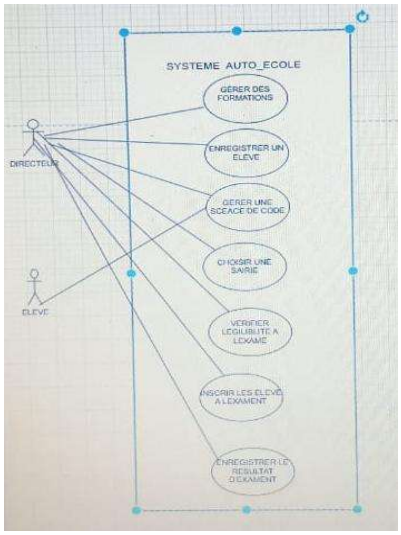
```
        self.__binomes: list[Binome] = []

    @property
    def date_remise_prevue(self) -> date:
        return self.__date_remise_prevue

    def ajouter_binome(self, binome: "Binome") -> None:
        self.__binomes.append(binome)

    def __str__(self) -> str:
        return f"Projet({self.__matiere}, {self.__sujet})"

class Binome:
    def __init__(self, numero_binome: int, projet: Projet, note_rapport: float, date_remise_effective:
date) -> None:
        self.__numero_binome = numero_binome
        self.__projet = projet
        self.__note_rapport = note_rapport
        self.__date_remise_effective = date_remise_effective
        self.__membres: list[Etudiant] = []
        projet.ajouter_binome(self)

    @property
    def note_rapport(self) -> float:
        return self.__note_rapport

    @property
    def membres(self) -> list[Etudiant]:
        return list(self.__membres)

    def ajouter_membre(self, etudiant: Etudiant) -> None:
        if len(self.__membres) >= 2:
```

```
            raise ValueError("Un binome ne peut pas depasser 2 membres.")
        self.__membres.append(etudiant)

    def calculer_penalite(self, points_par_jour: float = 1.0) -> float:
        retard = (self.__date_remise_effective - self.__projet.date_remise_prevue).days
        return max(0, retard) * points_par_jour

    def __str__(self) -> str:
        return f"Binome {self.__numero_binome}"

class EvaluationIndividuelle:
    def __init__(self, etudiant: Etudiant, binome: Binome, note_soutenance: float) -> None:
        if etudiant not in binome.membres:
            raise ValueError("L'etudiant doit appartenir au binome.")
        self.__etudiant = etudiant
        self.__binome = binome
        self.__note_soutenance = note_soutenance

    def calculer_note_finale(self) -> float:
        moyenne = (self.__binome.note_rapport + self.__note_soutenance) / 2
        note = moyenne - self.__binome.calculer_penalite()
        return max(0.0, note)

    def __str__(self) -> str:
        return f"Evaluation({self.__etudiant}, note finale={self.calculer_note_finale():.2f})"

if __name__ == "__main__":
    formation = Formation(1, "MIAGE-IF", "Initiale")
    etudiant1 = Etudiant(101, "Diop", "Aminata", formation)
    etudiant2 = Etudiant(102, "Fall", "Moussa", formation)
    projet = Projet(1, "UML", "Gestion des projets", date(2026, 4, 10))
```

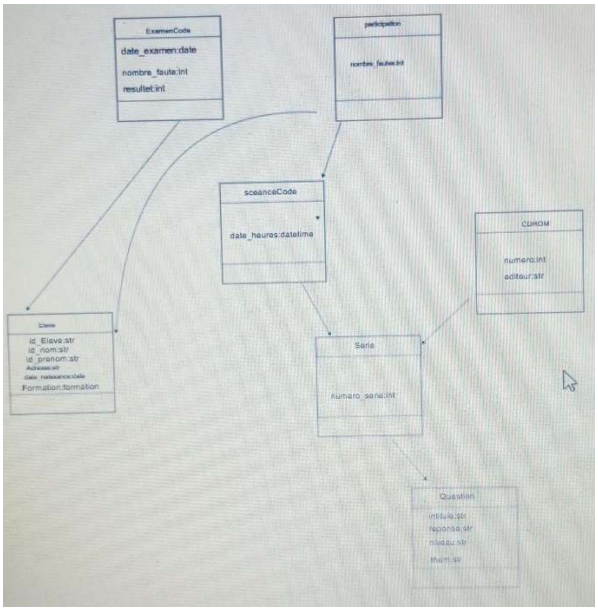
```
binome = Binome(1, projet, 16.0, date(2026, 4, 12))
binome.ajouter_membre(etudiant1)
binome.ajouter_membre(etudiant2)
print(EvaluationIndividuelle(etudiant1, binome, 14.0))
print(EvaluationIndividuelle(etudiant2, binome, 15.0))
```

SUJET2

1) Diagramme de cas d'utilisation



2)**diagramme de classe**



3)**Code python**

```
from __future__ import annotations
```

```
from datetime import date, datetime


class Eleve:
    def __init__(self, id_eleve: int, nom: str, prenom: str, adresse: str, date_naissance: date) -> None:
        self.__id_eleve = id_eleve
        self.__nom = nom
        self.__prenom = prenom
        self.__adresse = adresse
        self.__date_naissance = date_naissance
        self.__participations: list[Participation] = []

    def ajouter_participation(self, participation: "Participation") -> None:
        self.__participations.append(participation)

    def est_eligible_examen(self) -> bool:
        dernieres = sorted(self.__participations, key=lambda p: p.seance.date_heure, reverse=True)[:4]
        return len(dernieres) == 4 and all(p.nombre_fautes <= 5 for p in dernieres)

    def __str__(self) -> str:
        return f"{self.__prenom} {self.__nom}"

class CDROM:
    def __init__(self, numero: int, editeur: str) -> None:
        self.__numero = numero
        self.__editeur = editeur
        self.__series: list[Serie] = []

    def ajouter_serie(self, serie: "Serie") -> None:
        self.__series.append(serie)

class Question:
```

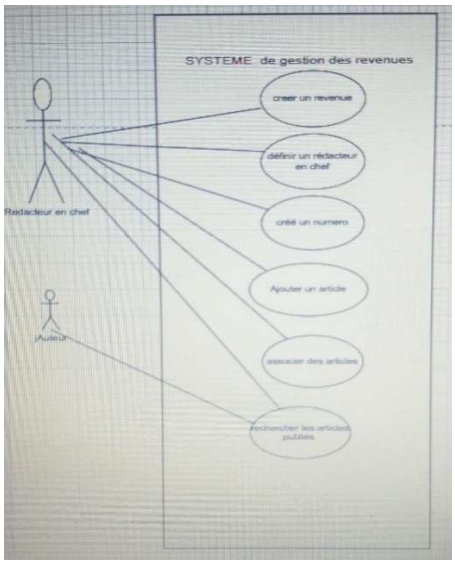
```
    def __init__(self, intitule: str, reponse: str, niveau: str, theme: str) -> None:
        self.__intitule = intitule
        self.__reponse = reponse
        self.__niveau = niveau
        self.__theme = theme

class Serie:
    def __init__(self, numero_serie: int, cdrom: CDROM) -> None:
        self.__numero_serie = numero_serie
        self.__cdrom = cdrom
        self.__questions: list[Question] = []
        cdrom.ajouter_serie(self)

    def ajouter_question(self, question: Question) -> None:
        self.__questions.append(question)

class SeanceCode:
    def __init__(self, date_heure: datetime, serie: Serie) -> None:
        self.__date_heure = date_heure
        self.__serie = serie

    @property
    def date_heure(self) -> datetime:
        return self.__date_heure

class Participation:
    def __init__(self, eleve: Eleve, seance: SeanceCode, nombre_fautes: int) -> None:
        if not 0 <= nombre_fautes <= 40:
            raise ValueError("Le nombre de fautes doit etre compris entre 0 et 40.")
        self.__eleve = eleve
        self.__seance = seance
```

```
        self.__nombre_fautes = nombre_fautes
        eleve.ajouter_participation(self)

    @property
    def seance(self) -> SeanceCode:
        return self.__seance

    @property
    def nombre_fautes(self) -> int:
        return self.__nombre_fautes

class ExamenCode:
    capacite_max = 8
    inscriptions_par_date: dict[date, list["ExamenCode"]] = {}

    def __init__(self, eleve: Eleve, date_examen: date, nombre_fautes: int) -> None:
        if not eleve.est_eligible_examen():
            raise ValueError("L'eleve n'est pas eligible a l'examen.")
        inscriptions = self.inscriptions_par_date.setdefault(date_examen, [])
        if len(inscriptions) >= self.capacite_max:
            raise ValueError("Capacite maximale atteinte pour cette date.")
        self.__eleve = eleve
        self.__date_examen = date_examen
        self.__nombre_fautes = nombre_fautes
        inscriptions.append(self)

    @property
    def resultat(self) -> str:
        return "admis" if self.__nombre_fautes <= 5 else "recale"

    def __str__(self) -> str:
```

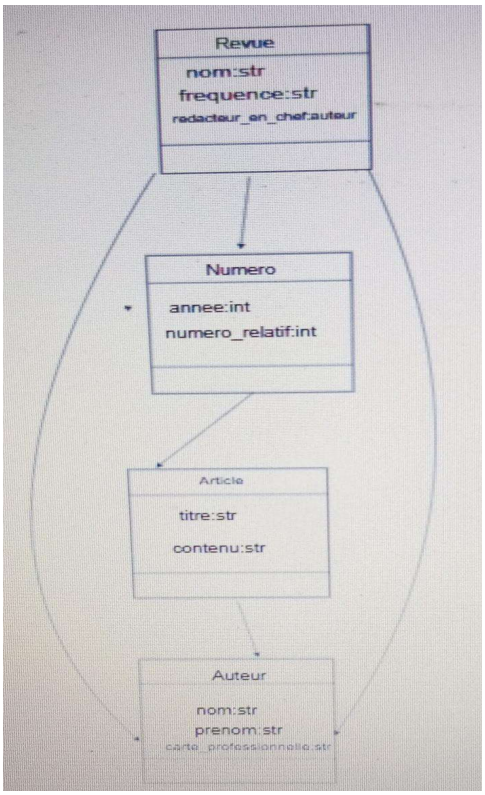
```
        return f"Examen({self.__date_examen}, {self.resultat})"


if __name__ == "__main__":
    eleve = Eleve(1, "Ndiaye", "Fatou", "Dakar", date(2000, 5, 12))
    cdrom = CDROM(14, "Code Expert")
    serie = Serie(5, cdrom)
    serie.ajouter_question(Question("Question 1", "Oui", "Moyen", "Signalisation"))
    Participation(eleve, SeanceCode(datetime(2026, 4, 1, 10, 0), serie), 4)
    Participation(eleve, SeanceCode(datetime(2026, 4, 3, 10, 0), serie), 5)
    Participation(eleve, SeanceCode(datetime(2026, 4, 5, 10, 0), serie), 3)
    Participation(eleve, SeanceCode(datetime(2026, 4, 7, 10, 0), serie), 2)
    print(eleve)
    print("Eligible:", eleve.est_eligible_examen())
    print(ExamenCode(eleve, date(2026, 4, 20), 4))
```

Sujet3

1)digramme de cas d'utilisation



2)diagramme de classe



2) Code python

```
from __future__ import annotations


class Auteur:
    def __init__(self, nom: str, prenom: str, carte_professionnelle: str, revue_employeur: "Revue") -> None:
        self.__nom = nom
        self.__prenom = prenom
        self.__carte_professionnelle = carte_professionnelle
        self.__revue_employeur = revue_employeur

    @property
    def nom_complet(self) -> str:
        return f"{self.__prenom} {self.__nom}"

    def __str__(self) -> str:
        return self.nom_complet
```

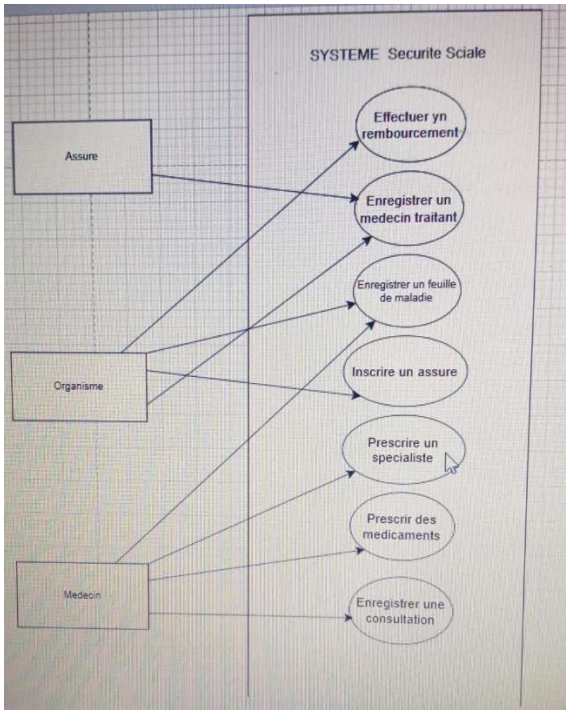
```
class Article:
    def __init__(self, titre: str, contenu: str) -> None:
        self.__titre = titre
        self.__contenu = contenu
        self.__auteurs: list[Auteur] = []

    @property
    def titre(self) -> str:
        return self.__titre

    @property
    def auteurs(self) -> list[Auteur]:
        return list(self.__auteurs)

    def ajouter_auteur(self, auteur: Auteur) -> None:
        if auteur not in self.__auteurs:
            self.__auteurs.append(auteur)

class Numero:
    def __init__(self, revue: "Revue", annee: int, numero_relatif: int) -> None:
        self.__revue = revue
        self.__annee = annee
        self.__numero_relatif = numero_relatif
        self.__articles: list[Article] = []
        revue.ajouter_numero(self)

    def ajouter_article(self, article: Article) -> None:
        for numero in self.__revue.numeros:
            if numero is not self and article in numero.articles:
                raise ValueError("Un article ne peut pas apparaitre dans plusieurs numeros de la
meme revue.")
        self.__articles.append(article)
```

```
    @property
    def articles(self) -> list[Article]:
        return list(self.__articles)

    def lister_auteurs(self) -> list[str]:
        noms = {auteur.nom_complet for article in self.__articles for auteur in article.auteurs}
        return sorted(noms)

    def __str__(self) -> str:
        return f"Numero {self.__numero_relatif}/{self.__annee}"

class Revue:
    def __init__(self, nom: str, frequence: str) -> None:
        self.__nom = nom
        self.__frequence = frequence
        self.__redacteur_en_chef: Auteur | None = None
        self.__numeros: list[Numero] = []

    @property
    def numeros(self) -> list[Numero]:
        return list(self.__numeros)

    def definir_redacteur_en_chef(self, auteur: Auteur) -> None:
        self.__redacteur_en_chef = auteur

    def ajouter_numero(self, numero: Numero) -> None:
        self.__numeros.append(numero)

    def __str__(self) -> str:
        return f"Revue({self.__nom})"
```

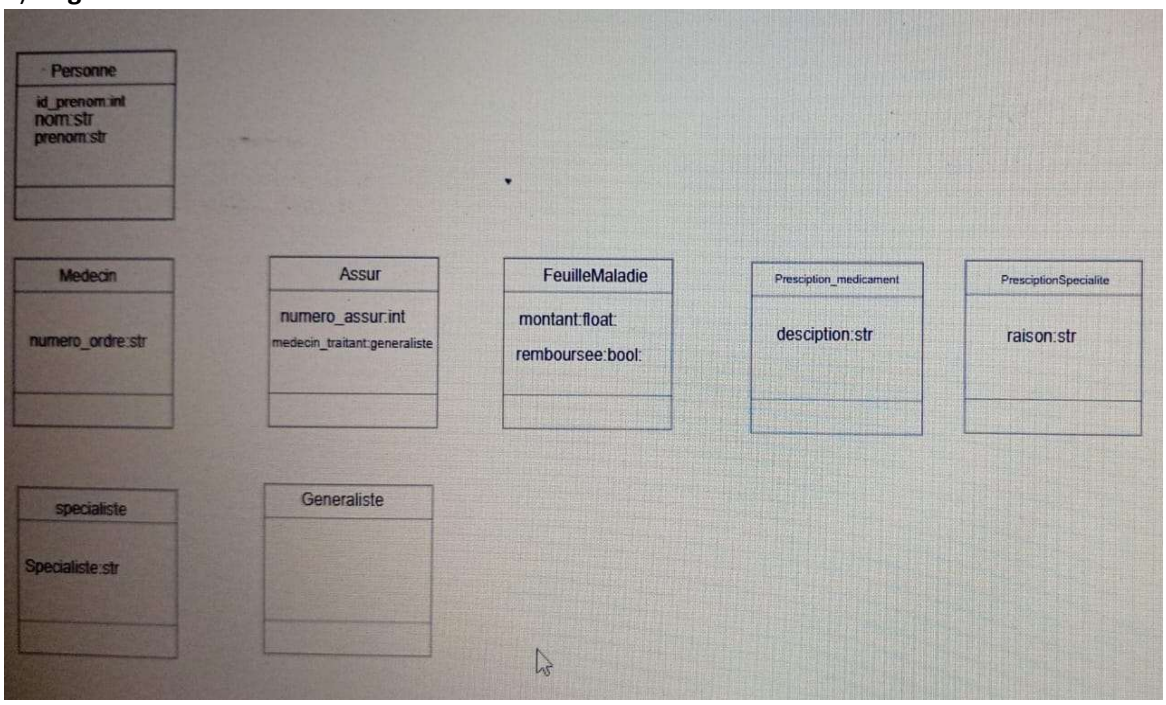
```
if __name__ == "__main__":
    revue = Revue("Linux Magazine", "mensuelle")
    auteur1 = Auteur("Ba", "Ibrahima", "CP001", revue)
    auteur2 = Auteur("Sarr", "Awa", "CP002", revue)
    revue.definir_redacteur_en_chef(auteur1)
    article = Article("Introduction a Python", "Contenu")
    article.ajouter_auteur(auteur1)
    article.ajouter_auteur(auteur2)
    numero = Numero(revue, 2026, 3)
    numero.ajouter_article(article)
    print(revue)
    print(numero)
    print("Auteurs:", numero.lister_auteurs())
```

EXO4

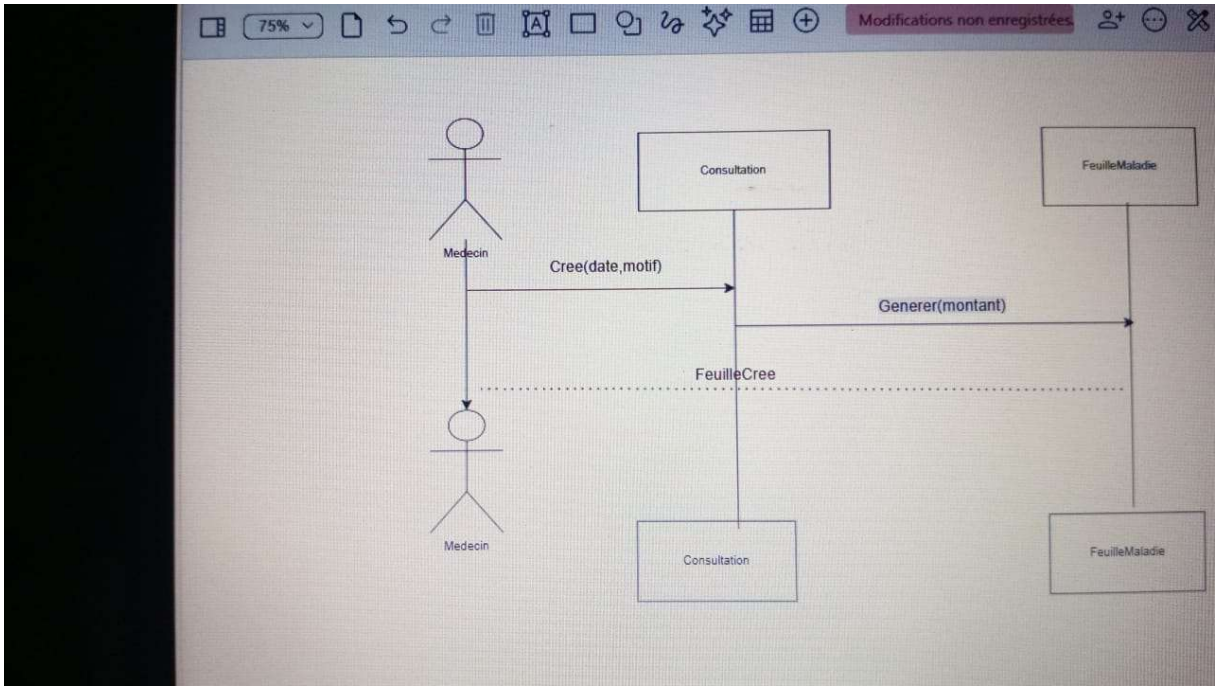
1)**Digramme de cas d'utilisation**



2)**Diagramme de classe**



3)diagramme de séquence



3) Code python

```
4) from __future__ import annotations
5)
6) from datetime import date
7)

8) class Personne:
9)     def __init__(self, id_personne: int, nom: str, prenom: str) -> None:
```

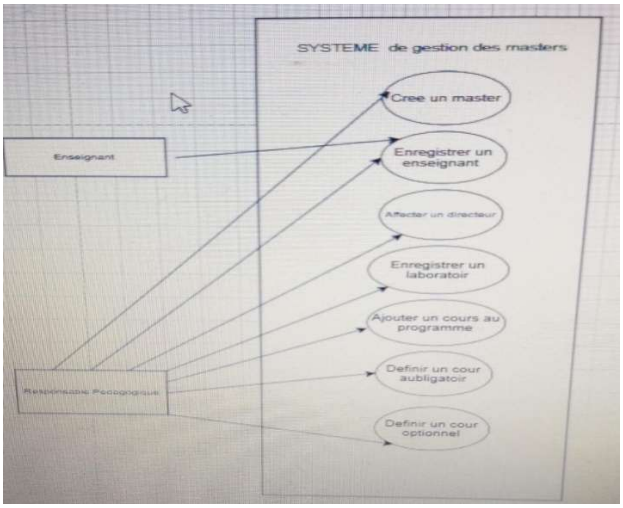
```
10)         self._id_personne = id_personne
11)         self._nom = nom
12)         self._prenom = prenom
13)
14)     def __str__(self) -> str:
15)         return f"{self._prenom} {self._nom}"
16)

17)class Assure(Personne):
18)     def __init__(self, id_personne: int, nom: str, prenom: str, nu-
   mero_assure: str) -> None:
19)         super().__init__(id_personne, nom, prenom)
20)         self.__numero_assure = numero_assure
21)         self.__medecin_traitant: Generaliste | None = None
22)
23)     def definir_medecin_traitant(self, medecin: "Generaliste") -> None:
24)         self.__medecin_traitant = medecin
25)

26)class Medecin(Personne):
27)     def __init__(self, id_personne: int, nom: str, prenom: str, nu-
   mero_ordre: str) -> None:
28)         super().__init__(id_personne, nom, prenom)
29)         self.__numero_ordre = numero_ordre
30)

31)class Generaliste(Medecin):
32)     pass
33)

34)class Specialiste(Medecin):
35)     def __init__(self, id_personne: int, nom: str, prenom: str, nu-
   mero_ordre: str, specialite: str) -> None:
36)         super().__init__(id_personne, nom, prenom, numero_ordre)
37)         self.__specialite = specialite
38)

39)class PrescriptionMedicament:
40)     def __init__(self, description: str) -> None:
41)         self.__description = description
42)

43)class PrescriptionSpecialiste:
44)     def __init__(self, specialiste: Specialiste, raison: str) -> None:
45)         self.__specialiste = specialiste
46)         self.__raison = raison
47)
48)     def __str__(self) -> str:
49)         return f"Consulter {self.__specialiste} pour {self.__raison}"
50)
```

```python
class FeuilleMaladie:
    def __init__(self, montant: float) -> None:
        self.__montant = montant
        self.__remboursee = False

    def rembourser(self) -> None:
        self.__remboursee = True

    def __str__(self) -> str:
        statut = "remboursee" if self.__remboursee else "en attente"
        return f"Feuille({self.__montant}, {statut})"

class Consultation:
    def __init__(self, patient: Assure, generaliste: Generaliste, date_consultation: date, motif: str, montant: float) -> None:
        self.__patient = patient
        self.__generaliste = generaliste
        self.__date_consultation = date_consultation
        self.__motif = motif
        self.__prescriptions_medicaments: list[PrescriptionMedicament] = []
        self.__prescriptions_specialistes: list[PrescriptionSpecialiste] = []
        self.__feuille_maladie = FeuilleMaladie(montant)

    @property
    def feuille_maladie(self) -> FeuilleMaladie:
        return self.__feuille_maladie

    def prescrire_medicament(self, description: str) -> None:
        self.__prescriptions_medicaments.append(PrescriptionMedicament(description))

    def prescrire_specialiste(self, specialiste: Specialiste, raison: str) -> None:
        self.__prescriptions_specialistes.append(PrescriptionSpecialiste(specialiste, raison))

    def __str__(self) -> str:
        return f"Consultation({self.__date_consultation}, {self.__motif})"

if __name__ == "__main__":
    patient = Assure(1, "Diallo", "Mariama", "AS100")
    generaliste = Generaliste(2, "Ndiaye", "Mamadou", "MED001")
    specialiste = Specialiste(3, "Sow", "Aissatou", "MED002", "Cardiologie")
```

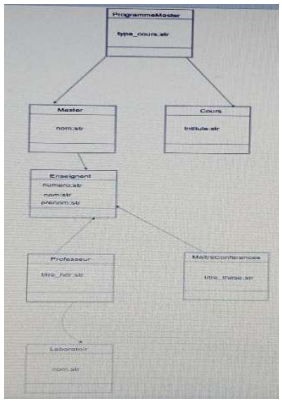
```
90)     patient.definir_medecin_traitant(generaliste)
91)     consultation = Consultation(patient, generaliste, date(2026, 4, 15),
    "Douleur thoracique", 25000.0)
92)     consultation.prescrire_medicament("Paracetamol")
93)     consultation.prescrire_specialiste(specialiste, "controle car-
    diaque")
94)     consultation.feuille_maladie.rembourser()
95)     print(patient)
96)     print(consultation)
97)     print(consultation.feuille_maladie)
98)
```

EXO5

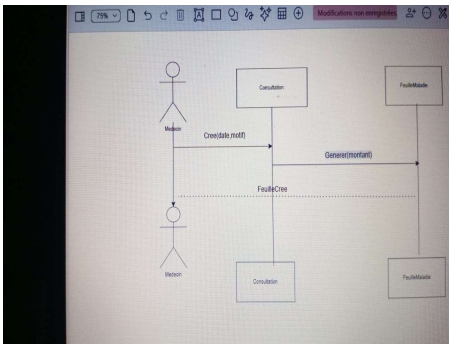
1) Diagramme de cas d'utilisation



2) Diagramme de classe



3)Diagramme de sequence



4)code python

```
from __future__ import annotations


class Enseignant:
    def __init__(self, numen: str, nom: str, prenom: str) -> None:
        self.__numen = numen
        self.__nom = nom
        self.__prenom = prenom

    def __str__(self) -> str:
        return f"{self.__prenom} {self.__nom}"


class Professeur(Enseignant):
    def __init__(self, numen: str, nom: str, prenom: str, titre_hdr: str) -> None:
        super().__init__(numen, nom, prenom)
        self.__titre_hdr = titre_hdr
        self.__laboratoire: Laboratoire | None = None

    def definir_laboratoire(self, laboratoire: "Laboratoire") -> None:
        self.__laboratoire = laboratoire


class MaitreConferences(Enseignant):
    def __init__(self, numen: str, nom: str, prenom: str, titre_these: str) -> None:
        super().__init__(numen, nom, prenom)
        self.__titre_these = titre_these
```

```
class Laboratoire:
    def __init__(self, nom: str) -> None:
        self.__nom = nom

    def __str__(self) -> str:
        return self.__nom

class Cours:
    def __init__(self, intitule: str) -> None:
        self.__intitule = intitule

    def __str__(self) -> str:
        return self.__intitule

class ProgrammeMaster:
    def __init__(self, cours: Cours, type_cours: str) -> None:
        if type_cours not in {"obligatoire", "optionnel"}:
            raise ValueError("Le type doit etre obligatoire ou optionnel.")
        self.__cours = cours
        self.__type_cours = type_cours

    def __str__(self) -> str:
        return f"{self.__cours} ({self.__type_cours})"

class Master:
    def __init__(self, nom: str, directeur: Enseignant) -> None:
        self.__nom = nom
        self.__directeur = directeur
        self.__programme: list[ProgrammeMaster] = []
```

```
    def ajouter_cours(self, cours: Cours, type_cours: str) -> None:
        self.__programme.append(ProgrammeMaster(cours, type_cours))

    def afficher_programme(self) -> list[str]:
        return [str(item) for item in self.__programme]

    def __str__(self) -> str:
        return f"Master({self.__nom})"

if __name__ == "__main__":
    professeur = Professeur("NUM001", "Kane", "Omar", "HDR Informatique")
    laboratoire = Laboratoire("Labo IA")
    professeur.definir_laboratoire(laboratoire)
    master = Master("Master Informatique", professeur)
    master.ajouter_cours(Cours("UML"), "obligatoire")
    master.ajouter_cours(Cours("Data Mining"), "optionnel")
    print(professeur)
    print(master)
    print(master.afficher_programme())
```